

Family League

Business Requirements

Version 1.0

1. Purpose

- This document describes the business needs of the intended platform.

Platform Definition

- Intended and limited to Service (Backend) end of the platform (User Interface is not in the scope)
- Platform is intended for
- Family and friends to predict the outcome of an actual league
- Gain points when the predictions match the actual result
- Support multiple such leagues
- League consists of series of matches and matches are played between two teams at a time
- Each team consists of n number of players
- League schedule is exact replica of actual league
- Predictions be at League level (Who will be at 1st, 2nd, 3rd positions as winners)
- Predictions are to be allowed to the full leaderboard (1 to n, n being the number of teams playing)
- Predictions are at Match level (Who wins the match, who wins the toss, who will be the player of the match)
- League predictions must close 4 hrs prior to the first match start time
- Match predictions must close 1 hr prior to the start time
- Results will be manually updated by Admin persona after the actual result is declared
- Leader board calculations needs to be performed in async mode and informed to admin via email
- Users must get emails to indicate prior to match closure if one hasn't opted the predictions
- Admin must get email alerts for Results updation
- Admin APIs to create entire set of League Information (API Collection and docs together should help the Admin Persona)
- APIs must be protected by Persona based access
- All the emails must be available in data store (When, to whom, for what, timestamp, status and so on)

- All the data changes must be captured as standard audit data capture
- Users are allowed to see each others predictions only after the prediction window is closed
- Prediction window is open from the moment League is started until closed by the system (as configured, 1 hr prior to start time)

2. Technical Needs

- A Spring Boot based backend web application
- PostgreSQL as data store
- Email notification.
- Scheduled email alerts
- All standard practices of backend development

3. Core Workflows

3.1 User, RBAC and ACL Management

3.1 Authentication and Authorization

3.2 Profile Updates (avatar name and profile pic for the avatar)

3.3 Match Prediction Flow

1. Match is created with schedule and lock time.
2. User submits predictions before lock.
3. System enforces lock n hours before the match starts.
4. After lock, no edits are allowed.
5. Head-to-head (among the users) becomes available only after lock.

3.4 Result Processing Flow

1. Admin explicitly publishes the result
2. Points are recalculated server-side
3. Leaderboard rankings are updated and notifications sent

3.5 League Flow

- Admin creates the league setup
- League predictions are open
- League predictions are locked 4 hrs prior to the first match start time
- League level leaderboard of team positions keep getting adjusted by each match result (Part of Match result processing)

- League level results will be updated by Admin
- Final Leaderboard is calculated and updated
- Admin verifies the results and Closes the league
- Closed leagues are accessible but with out amend capability (not even for Admin)
- Teams are independent of League season (Each team could play many leagues of Same name)
- This forces a definition of League as an umbrella and season as an instance of league

4. Backend Requirements

4.1 API Framework

- Use Spring Boot (Java) for all application APIs.
- use Dependency Injection
- Use Spring Security for authentication and role-based authorization.
- Use Spring Validation for request validation.

4.2 Persistence

- Use PostgreSQL (MySQL can also be chosen but justify, why)
- Use Spring Data JPA with Hibernate as the ORM
- Use Flyway for database migrations (Optionally)
- Use Spring Data repositories for data access patterns

4.4 Data Model

- Derive based on the functional need
- No records get deleted permanently, it only a soft delete unless needed (Need must be logged as a decision in decision log)

5. Notifications

- Admin should have ability to bulk communicate by choosing users and event types with custom messaging

6. Scoring Rules

- One Prediction add one point to the user score in the league
- Ranks will be allocated based on the total Points

6.1 Match Scoring

- Match winner, Player of the Match , Toss winner
- Ties count as official results and either side gets 1 point

6.3 Leaderboard

- Total points determine rank.
- Rankings recalculate after each confirmed result.

7. Security and Integrity

- HTTPS is needed (Local certs are fine)
- JWT sessions are required.
- Role-based authorization must protect admin routes.
- Prediction lock must be enforced at the database level.
- Points must never be accepted via API, system calculates (Not even by Admin).

Submission Checklist

What repo should have and should not have

- Submission preferably as github repo allowing Gopal and Rama as moderators
- README.md exists at the root and links to all other docs
- Repo should be assessed by the main branch only
- App starts successfully from a clean clone following the steps in README.md
- All the documentation within the repo only
- Composition of all the AI Prompts mentioning the AI Tool very clearly
- No hardcoded credentials, API keys, or personal data in the codebase

Must have

- Clear logging to the right level and detail (Console and File are must)
- Inline documentation should be apt
- Exception handling
- Consistent API agreements (Request and Response Data Structures)
- Pagination, Search, Sort all handled in the APIs that needs these
- Interface driven development where applicable
- API Collection in any form chosen including OpenAPI documentation
- Authentication and Authorization (Simple Auth, if OAUTH chosen, that is also fine)

- JWT based token management
- Clear data model (Very detailed model should exist)
- Decision log with justifications
- Standards of Java, JEE, Spring and in general backend development good practices
- Time driven notifications must be through scheduling
- Configurability to the max
- Modularity to the cleanest possible limits
- Normalisation to the best
- Any decision made to decision log with justification
- APIs must be apt to the business needs (all needs by one API is not acceptable)

Good to have

- Google or other OAUTH based authentication
- Abstracting the common attributes in ORM
- Auto population of audit fields (created_at, created_by, updated_at and so on)
- Unit Testing
- e2e Testing of APIs
- Batch Processing (example sending emails, or prediction score calculations)
- Caching (Only if safe guarded with proper eviction policy)